Case Study: Facilitating a Requirements Deep-Dive on Cimply's PRD

Jynae Myklebust | Technical Writer / Documentation & Requirements Specialist

Writing sample — Cimply is a demonstration scheduling product built to showcase documentation and process skills. Not a live commercial product.

THE SITUATION

Cimply's PRD had already been through one revision cycle when I opened a session to reconcile it against a decision I remembered making in a separate conversation: that Cimply needed a third role, distinct from Scheduler and Designer, responsible for managing who's on the account. I asked whether that decision had actually made it into the document. It hadn't — the word “admin” appeared exactly twice, both describing status configuration, with nothing about invites, roles, or account management. That gap became the first, and largest, piece of work in the session.

SETTING THE OPERATING MODE

Once the Admin / User Management section was designed and the PRD updated, I had a running list of smaller inconsistencies to work through, and I didn't want to relitigate my process for every one of them. I asked directly: does it make more sense to review each one together, or make the small changes myself and have them reviewed after? The answer set the rule for the rest of the session — anything self-contained goes straight in; anything with ripple effects across statuses, roles, or the data model gets talked through first. I used that rule for roughly twenty more items, and it held up: everything that actually needed discussion turned out to touch more than one part of the document.

THE BIG DESIGN CALL: ADMIN AND USER MANAGEMENT

Designing Admin from nothing meant making calls with real consequences for the rest of the schema, not just adding a checkbox to a user record.

- Delete vs. suspend. Hard-deleting a user conflicts with a principle already in the PRD — every order and message is permanently attributed to whoever touched it. I settled on suspension as the only offboarding mechanism for anyone who's ever logged in, with true delete reserved for a still-pending invite, since a pending user has no history to protect in the first place.
- What happens to a suspended Designer's open orders. Two options were on the table — manually reassign them, or auto-bump their status to Needs Review so someone would look at them. I rejected the auto-bump: a status is supposed to describe what actually happened to the work, not just get someone's attention. I flagged the existing Past-Due visual-flag mechanism as a better fit, then simplified it further than what was proposed — applying the flag uniformly to any non-terminal order rather than writing different rules per status. Whether it gets reassigned or just flagged for awareness is the scheduler's call, not the system's.
- No separate Owner tier. It's a common pattern — a non-revocable role tied to billing, distinct from Admin, so an account can never lose all administrative access. I turned it down for MVP: Cimply has no billing yet, and the last-admin-protection rule already prevents the failure mode a separate Owner tier exists to solve.

WHERE MY JUDGMENT DIVERGED FROM THE DRAFT

- Broadened a scheduler-reassignment rule that had been scoped narrowly — reassignable only when the current scheduler is suspended — to apply regardless of suspension status, since there's no reason ownership should be locked to one person for any cause.
- Introduced a real scope expansion mid-review: that a Scheduler should also be eligible to be assigned as a Designer on an order, since small studios often have one person doing both jobs. This wasn't a wording change — several requirements written as “Designer does X” needed to become “whoever is assigned does X,” shifting permission logic from role-gated to assignment-gated across the document.
- Declined to add a guardrail blocking an Admin from removing their own flag outright, even when they're not the account's last Admin — there wasn't a real failure mode it would prevent, and rules without a reason behind them aren't good practice either.
- Reframed a binary between splitting first/last name fields and keeping one combined field into a third option: a single display_name field now, with room to split later only if a real reporting need shows up for it.

CATCHING A STRUCTURAL BUG

The most consequential catch in the session came from rereading my own document, not from a new feature request. If a Designer has to be assigned at order creation, the Assigned status doesn't actually mean anything distinct from New — there's no way to be in one without immediately being in the other. Making designer assignment optional at creation fixed that: the assigned_to field had to become nullable, order creation and order assignment had to be treated as genuinely separate events, and — as a side effect — it unblocked a future feature (designers self-assigning from a pool of unassigned orders) that had no path to working under the old model. One clarifying question about status logic turned into a schema change and unlocked a future feature at the same time.

SCOPING CALLS: WHAT MOVED, WHAT STAYED, WHAT GOT DEFERRED

Not every gap belongs in v1, and knowing which is which was most of the work in the second half of the session:

- The Late Start flag went straight into the current release, not a later one, because it reuses a flag mechanism the document already had — the cost to add it was close to zero, so there was no reason to make people wait for it.
- Data export got written up as an explicit v1 Non-Goal and pushed to a later release, tied directly back to a goal already in the document (using logged time for billing or capacity planning), so it doesn't get lost as an orphaned nice-to-have.
- SMS notifications got scoped down from an open-ended ask to one-way only, once it was clear two-way SMS meant real regulatory and infrastructure weight that had no business sharing a decision with “notify me by text.”
- Order/Work Type overlap detection — a gap I raised myself, with concrete examples of exclusive versus stackable work types — went to a future release as a full concept, but I still nailed down the near-term behavior rules now (warn, don't block; only flag active orders) rather than leaving the whole thing undefined.

- Backlog priority ordering I left alone on purpose. Ranking items before there's real usage data or engineering estimates behind them creates a false sense of authority — better to leave it unranked until there's something to base the ranking on.

WHY THIS MATTERS

None of this was about writing clean sentences — it was product judgment applied at the level of a single field name, a single status transition, a single deferred flag. Working this way with an assistant means the tool can propose options and catch inconsistencies I hadn't consciously spotted — a status-name collision, an orphaned-record risk, three similar situations each being handled differently — but the calls, what's in scope, what's a Non-Goal, what needs a guardrail and what doesn't, stayed mine throughout. That's the working style I want visible here: I can move fast through a requirements review using modern tooling, and the document that comes out the other side still reflects deliberate decisions, not just whatever the first draft suggested.